

LẬP TRÌNH PYTHON CHO MACHINE LEARNING

Từ Python cơ bản đến Machine Learning

Ngô Hoàng Dũng - Lê Hoàng Ân - Nguyễn Thị Hà - Lê Ngọc Ánh Dương

Ngày 7 tháng 9 năm 2026

Mục lục

1	Giới thiệu về Python	1
1.1	Python là gì?	1
1.2	Lịch sử phát triển của Python	1
1.2.1	Khởi nguồn & Phiên bản thử nghiệm (1989 – 1991)	1
1.2.2	Python 1.0 & Chuẩn hóa quản trị (1994 – 1999)	1
1.2.3	Python 2.0 & Sự trỗi dậy của Cộng đồng (Tháng 10/2000)	2
1.2.4	Python 3.0 – ”Cuộc đại tái thiết” (Tháng 12/2008)	2
1.2.5	Kỷ nguyên AI/Machine Learning & Khai tử Python 2 (2015 – Nay)	2
1.3	Guido van Rossum – tác giả của Python	2
1.4	Tên gọi Python	2
1.5	Hướng dẫn cài đặt Python và Anaconda	2
1.5.1	Cài đặt Python	2
1.5.2	Cài đặt Anaconda/Miniconda	3
1.5.3	Tạo môi trường Conda	4
1.6	Tóm tắt chương	4
2	Cú pháp Python cơ bản	5
2.1	Biến và Kiểu dữ liệu	5
2.1.1	Các kiểu dữ liệu cơ bản	5
2.2	Cấu trúc dữ liệu nâng cao	5
2.2.1	List (Danh sách)	6
2.3	Cấu trúc điều khiển và Vòng lặp	6
2.3.1	Câu lệnh điều kiện if-else	6
2.4	Tóm tắt chương	6
3	Lập trình hướng đối tượng trong Python	7
3.1	Lớp và Đối tượng (Class and Object)	7
3.2	Các phương thức đặc biệt (Magic Methods)	7
3.2.1	Phương thức <code>__init__</code> và <code>__call__</code>	7
3.3	Tính Kế thừa (Inheritance)	8
3.4	Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters)	9
3.5	Thiết kế Custom Estimator chuẩn Scikit-Learn	9
3.6	Mẫu thiết kế Pipeline	10
3.7	Tóm tắt chương	10

4	Hồi quy tuyến tính (Linear Regression)	11
4.1	Mô hình Hồi quy tuyến tính đơn biến	11
4.2	Hàm mất mát (Loss Function)	11
4.3	Cài đặt Hồi quy tuyến tính với Scikit-Learn	11
4.4	Tóm tắt chương	12

Chương 1: Giới thiệu về Python

1.1 Python là gì?

Python là một ngôn ngữ lập trình được sử dụng rộng rãi trong các ứng dụng web, phát triển phần mềm, khoa học dữ liệu và máy học (ML). Các nhà phát triển sử dụng Python vì nó hiệu quả, dễ học và có thể chạy trên nhiều nền tảng khác nhau. Phần mềm Python được tải xuống miễn phí, tích hợp tốt với tất cả các loại hệ thống và tăng tốc độ phát triển. (Nguồn: <https://aws.amazon.com/vi/what-is/python/>)

Python hỗ trợ nhiều mô hình lập trình, trong đó có lập trình thủ tục, lập trình hướng đối tượng và lập trình hàm.

Một chương trình Python đơn giản:

```
1 print("Hello , Python!")
```

Cú pháp của Python có xu hướng sử dụng khoảng trắng để thể hiện cấu trúc của chương trình. Ví dụ:

```
1 if age >= 18:
2     print("Adult")
3 else:
4     print("Under 18")
```

Cách tổ chức này giúp mã nguồn Python có cấu trúc rõ ràng và dễ đọc.

1.2 Lịch sử phát triển của Python

1.2.1 Khởi nguồn & Phiên bản thử nghiệm (1989 – 1991)

Vào cuối năm 1989, khi đang làm việc tại Viện Nghiên cứu Quốc gia về Toán học và Khoa học Máy tính (CWI) ở Hà Lan, Guido van Rossum quyết định viết một ngôn ngữ lập trình mới trong kỳ nghỉ Giáng sinh để làm dự án phụ. Ông muốn tạo ra một ngôn ngữ kết hợp tính đơn giản của ngôn ngữ ABC với khả năng tương tác tốt với hệ điều hành Amoeba.

Tháng 2/1991, Guido đăng tải mã nguồn lên nhóm tin alt.sources với phiên bản Python 0.9.0. Dù chỉ là bản phôi thai, nó đã có những nền móng rất hiện đại: Mô hình OOP, Hệ thống xử lý ngoại lệ, Các kiểu dữ liệu cốt lõi (list, dict, str) và Hệ thống module.

1.2.2 Python 1.0 & Chuẩn hóa quản trị (1994 – 1999)

Tháng 1/1994, Python 1.0 chính thức ra mắt, tích hợp mạnh mẽ các công cụ của Lập trình chức năng (Functional Programming) như lambda, map, filter và reduce.

Năm 1995 (Python 1.2), Guido tiếp tục phát triển Python tại Tập đoàn Sáng kiến Nghiên cứu Quốc gia (CNRI) ở Mỹ, bổ sung khả năng truyền tham số theo tên (Keyword arguments). Trong giai đoạn này, Guido được cộng đồng trao danh hiệu BDFL (Benevolent Dictator for Life).

1.2.3 Python 2.0 & Sự trỗi dậy của Cộng đồng (Tháng 10/2000)

Python 2.0 đánh dấu sự chuyển dịch lớn sang mô hình phát triển mã nguồn mở hoàn toàn. Các tính năng đột phá bao gồm: List Comprehension, Garbage Collector (GC), và ra đời quy trình PEP (Python Enhancement Proposal). Phiên bản Python 2.7 (2010) trở thành phiên bản ổn định nhất của dòng Python 2.

1.2.4 Python 3.0 – ”Cuộc đại tái thiết” (Tháng 12/2008)

Sau nhiều năm phát triển, Python 2 bị tích tụ nhiều lỗi thiết kế. Guido quyết định phát hành Python 3.0 với đặc điểm cốt lõi là Không tương thích ngược (Backwards Incompatible) với Python 2. Những cải tiến quan trọng bao gồm:

- Đồng bộ Unicode cho mọi chuỗi ký tự.
- Thay đổi câu lệnh `print` thành hàm `print()`.
- Tối ưu hóa bộ nhớ bằng cách chuyển các hàm như `range()`, `zip()`, `map()` sang dạng trả về Iterator.

1.2.5 Kỷ nguyên AI/Machine Learning & Khai tử Python 2 (2015 – Nay)

Cú pháp đơn giản giúp Python trở thành lựa chọn hàng đầu trong Khoa học dữ liệu và AI với các thư viện như NumPy, Pandas, Scikit-Learn, TensorFlow, PyTorch.

Ngày 01/01/2020, Python 2 chính thức bị khai tử (EOL), đưa Python 3 trở thành tiêu chuẩn duy nhất. Giai đoạn hiện tại (Python 3.8 – 3.13+) tập trung vào bổ sung các toán tử mới và dự án Faster CPython nhằm tối ưu hóa hiệu năng.

1.3 Guido van Rossum – tác giả của Python

Guido van Rossum là một nhà khoa học máy tính người Hà Lan. Ông là người giữ quyền quyết định tối cao về định hướng kiến trúc của ngôn ngữ (BDFL) cho đến khi từ chức vào năm 2018.

1.4 Tên gọi Python

Tên gọi ”Python” không xuất phát từ loài rắn, mà lấy cảm hứng từ chương trình hài kịch yêu thích của ông trên đài BBC: ”Monty Python’s Flying Circus”.

1.5 Hướng dẫn cài đặt Python và Anaconda

1.5.1 Cài đặt Python

1. Truy cập trang chủ Python: <https://www.python.org/downloads/>



Hình 1.1: Guido van Rossum - Tác giả sáng lập Python. (Nguồn: Sauria Associates, LLC / FlickrVision)

2. Tải phiên bản cài đặt (Ví dụ: Python 3.14.7 cho Windows).
3. Mở file cài đặt. **Lưu ý quan trọng:** Nhớ tick vào ô "Add python.exe to PATH" trước khi bấm "Install Now".
4. Bấm "Close" khi cài đặt thành công. Có thể chọn "Disable path length limit" nếu cần.



Hình 1.2: Tick chọn Add python.exe to PATH trong quá trình cài đặt

1.5.2 Cài đặt Anaconda/Miniconda

Anaconda là một nền tảng phân phối Python và R phục vụ cho khoa học dữ liệu.

1. Truy cập trang tải Anaconda: <https://www.anaconda.com/download>
2. Tải bản cài đặt Anaconda Distribution hoặc Miniconda.
3. Mở file cài đặt, chọn "Just Me", chọn đường dẫn lưu (ví dụ: C:\miniconda3).

4. Trong phần Advanced Installation Options, có thể tick chọn "Register Miniconda3 as my default Python".
5. Nhấn "Install" và đợi quá trình cài đặt hoàn tất.

1.5.3 Tạo môi trường Conda

Sau khi cài đặt Anaconda/Miniconda, mở Anaconda Prompt và sử dụng lệnh sau để tạo một môi trường ảo (ví dụ tên là ai) với phiên bản Python 3.14:

```
conda create -n ai python=3.14 anaconda
```

Xác nhận tiến trình cài đặt các gói bằng cách gõ y (yes) khi được yêu cầu.

1.6 Tóm tắt chương

Trong chương này, chúng ta đã tìm hiểu:

- Khái niệm cơ bản về Python và ứng dụng của nó.
- Lịch sử hình thành chi tiết qua từng giai đoạn (từ Python 0.9.0 đến Python 3.x).
- Tiểu sử của tác giả Guido van Rossum và nguồn gốc tên gọi thú vị của Python.
- Các bước cài đặt môi trường lập trình cơ bản với Python và Anaconda.

Chương 2: Cú pháp Python cơ bản

Chương này trình bày các khái niệm nền tảng trong lập trình Python, bao gồm biến, kiểu dữ liệu, toán tử, cấu trúc điều khiển và các hàm toán học. Đây là các kiến thức bắt buộc trước khi đi vào xử lý dữ liệu và Machine Learning.

2.1 Biến và Kiểu dữ liệu

Trong Python, biến được khởi tạo tự động khi gán giá trị. Python là ngôn ngữ *dynamically typed* (định kiểu động).

Định nghĩa 2.1 (Biến trong Python). *Biến là tên gọi đại diện cho một vùng nhớ chứa dữ liệu. Giá trị của biến có thể thay đổi trong quá trình thực thi chương trình.*

2.1.1 Các kiểu dữ liệu cơ bản

Dựa trên nền tảng Python Basic, dưới đây là bảng tổng hợp các kiểu dữ liệu cốt lõi thường xuyên được sử dụng:

Tên	Kiểu dữ liệu	Ví dụ
str	Kiểu chuỗi (string)	a = 'Thien Thanh'
int	Kiểu số nguyên	a = 79
float	Kiểu số thực	a = 7.6
NoneType	Kiểu None	a = None
bool	Kiểu luận lý (True, False)	a = True
list	Kiểu danh sách	a = [3, 2, 1, 5]
tuple	Kiểu bộ	a = (1, 5, 3)
dict	Kiểu từ điển	b = {'one': 1, 'four': 4}
set	Kiểu tập hợp	a = {1, 3, 5}

Bảng 2.1: Các kiểu dữ liệu cơ bản trong Python

Để kiểm tra kiểu dữ liệu của một biến trong Python, ta sử dụng hàm `type()`:

Listing 2.1: Kiểm tra kiểu dữ liệu

```
1 a = 7
2 print(type(a)) # Output: <class 'int'>
```


2.2 Toán tử trong Python

Toán tử là thành phần không thể thiếu để thực hiện các phép tính từ cơ bản đến phức tạp trong các thuật toán.

2.2.1 Toán tử cơ bản

Tên toán tử	Ký hiệu	Ví dụ	Kết quả
Cộng	+	3 + 7	10
Trừ	-	7 - 2	5
Nhân	*	2 * 7	14
Chia	/	7 / 5	1.4
Chia lấy phần nguyên	//	10 // 4	2
Chia lấy phần dư	%	9 % 5	4
Lấy mũ	**	2 ** 3	8

Bảng 2.2: Các toán tử số học cơ bản

2.2.2 Toán tử gán mở rộng

Để mã nguồn ngắn gọn hơn, Python hỗ trợ các toán tử gán mở rộng kết hợp phép tính và phép gán:

Tên toán tử	Ký hiệu	Ví dụ	Kết quả
Cộng	+=	a = 7; a += 3	10
Trừ	-=	b = 9; b -= 4	5
Nhân	*=	a = 2; a *= 7	14
Chia	/=	a = 7; b /= 5	1.4
Chia lấy phần nguyên	//=	a = 10; a //= 4	2
Chia lấy phần dư	%=	a = 9; a %= 5	4
Lấy mũ	**=	a = 2; a **= 3	8

Bảng 2.3: Các toán tử gán mở rộng

2.3 Hàm toán học trong Python

Đối với các ứng dụng khoa học dữ liệu, thư viện math cung cấp các hằng số và hàm tính toán nâng cao. Để sử dụng, ta cần import thư viện:

Listing 2.2: Import thư viện math

```
1 import math
```

2.3.1 Các hằng số toán học

- Hằng số Euler (E): `math.e` $\approx$ 2.718
- Hằng số Pi (π): `math.pi` $\approx$ 3.14159

2.3.2 Các hàm toán học thường dùng

Tên hàm (Toán học)	Mã lệnh Python
$\ln(x)$	<code>math.log(math.e)</code>
$\log_2(x)$	<code>math.log2(3)</code>
e^x	<code>math.exp(3)</code>
$\log_{10}(x)$	<code>math.log10(3)</code>
$\sqrt{x}$	<code>math.sqrt(3)</code>
a^x	<code>math.pow(2, 3)</code>

Bảng 2.4: Một số hàm toán học từ thư viện `math`

2.4 Cấu trúc dữ liệu nâng cao

Đối với Machine Learning, ngoài mảng 1 chiều, ta thường sử dụng 3 cấu trúc dữ liệu cơ bản bao gồm List, Tuple và Dictionary.

2.4.1 List (Danh sách)

List là tập hợp các phần tử có thứ tự và có thể thay đổi (mutable).

Listing 2.3: Thao tác cơ bản trên List

```
1 # Khởi tạo list
2 features = [1.2, 3.4, 5.1, 2.8]
3
4 # Truy cập và thay đổi phần tử
5 features[0] = 0.9
6 features.append(4.5)
7
8 print("List sau khi cập nhật:", features)
```

2.5 Cấu trúc điều khiển và Vòng lặp

2.5.1 Câu lệnh điều kiện if-else

Listing 2.4: Sử dụng cấu trúc điều kiện

```
1 score = 0.85
2
```

```
3 if score >= 0.8:  
4     print("Mô hình đạt độ chính xác cao")  
5 else:  
6     print("Cần can thiệp hoặc tối ưu thêm")
```

2.6 Tóm tắt chương

Trong chương này, chúng ta đã nắm được:

- Cách khai báo biến và chi tiết các kiểu dữ liệu cơ bản trong Python.
- Các toán tử số học và toán tử gán mở rộng.
- Cách sử dụng thư viện math cho các hàm toán học phức tạp.
- Cấu trúc dữ liệu nâng cao: List, Tuple, Dictionary.
- Cấu trúc rẽ nhánh trong Python.

Chương 3: Lập trình hướng đối tượng trong Python

Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là phương pháp luận lập trình đóng vai trò then chốt trong việc thiết kế các thư viện Machine Learning hiện đại như Scikit-Learn, PyTorch và TensorFlow. Chương này trình bày các khái niệm OOP cốt lõi và hướng dẫn cách ứng dụng OOP để tự đóng gói một mô hình Machine Learning.

3.1 Lớp và Đối tượng (Class and Object)

Định nghĩa 3.1 (Lớp và Đối tượng). *Lớp (Class)* là khuôn mẫu định nghĩa các thuộc tính (dữ liệu) và phương thức (hành vi). *Đối tượng (Object)* là một thể hiện cụ thể được khởi tạo từ lớp đó.

Ví dụ 3.1. Khai báo một lớp cơ bản lưu trữ mẫu dữ liệu:

Listing 3.1: Khai báo Class và tạo Object trong Python

```
1 class DataSample:
2     def __init__(self, feature_name, value):
3         self.feature_name = feature_name # Thuộc tính
4         self.value = value
5
6     def display_info(self):                # Phương thức
7         print(f"Feature: {self.feature_name}, Value: {self.value}")
8
9 # Khởi tạo đối tượng
10 sample = DataSample(feature_name="Age", value=25)
11 sample.display_info()
```

3.2 Các phương thức đặc biệt (Magic Methods)

Các phương thức có cú pháp hai dấu gạch dưới (`__method__`) giúp tùy biến hành vi của đối tượng trong Python.

3.2.1 Phương thức `__init__` và `__call__`

- `__init__`: Hàm khởi tạo (constructor), tự động thực thi khi tạo mới một đối tượng.

- `__call__`: Cho phép gọi trực tiếp một đối tượng giống như một hàm, thường dùng khi thiết kế các Layer trong Neural Network.

Listing 3.2: Sử dụng phương thức `__call__`

```

1 class Multiplier:
2     def __init__(self, factor):
3         self.factor = factor
4
5     def __call__(self, x):
6         return x * self.factor
7
8 # Su dụng như một hàm
9 double = Multiplier(factor=2)
10 print(double(5)) # Output: 10

```

3.3 Tính Kế thừa (Inheritance)

Tính kế thừa cho phép tạo một lớp mới dựa trên lớp đã có, giúp tái sử dụng mã nguồn và chuẩn hóa giao diện của các bộ biến đổi dữ liệu (Transformers).

Listing 3.3: Kế thừa lớp BaseTransformer trong tiền xử lý dữ liệu

```

1 class BaseTransformer:
2     def fit(self, arr):
3         raise NotImplementedError("Phương thức fit() bắt buộc phải
4             được triển khai ở lớp con.")
5
6     def transform(self, arr):
7         raise NotImplementedError("Phương thức transform() bắt buộc
8             phải triển khai ở lớp con.")
9
10    def fit_transform(self, arr):
11        self.fit(arr)
12        return self.transform(arr)
13
14 class MinMaxScaler(BaseTransformer):
15     def fit(self, arr):
16         self.min_ = min(arr)
17         self.max_ = max(arr)
18         return self
19
20     def transform(self, arr):
21         return [(x - self.min_) / (self.max_ - self.min_) for x in
22             arr]
23
24     def inverse_transform(self, arr):
25         return [x * (self.max_ - self.min_) + self.min_ for x in arr]

```

3.4 Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters)

Trong thiết kế Machine Learning, ta cần phân biệt rõ giữa siêu tham số (cấu hình trước khi huấn luyện) và tham số học được (được sinh ra sau khi gọi hàm fit).

Listing 3.4: Bắt lỗi an toàn khi quản lý tham số học được

```

1 class MinMaxScaler(BaseTransformer):
2     def __init__(self, feature_range=(0, 1)):
3         self.feature_range = feature_range # Hyperparameter
4
5     def fit(self, arr):
6         self.min = float(min(arr))          # Learned Parameter
7         self.max = float(max(arr))          # Learned Parameter
8         return self
9
10    def transform(self, arr):
11        # Kiểm tra an toàn: Đảm bảo mô hình đã được fit()
12        if not hasattr(self, 'min') or not hasattr(self, 'max'):
13            raise RuntimeError("Lỗi: Bạn phải gọi hàm fit() trước khi
14                               gọi transform()!")
15
16        low, high = self.feature_range
17        std = [(x - self.min) / (self.max - self.min) for x in arr]
18        return [x * (high - low) + low for x in std]

```

3.5 Thiết kế Custom Estimator chuẩn Scikit-Learn

Trọng tâm ứng dụng OOP trong Machine Learning là đóng gói thuật toán thành một Class có các giao diện chuẩn (fit, predict, evaluate). Dưới đây là triển khai mô hình Hồi quy tuyến tính đơn biến (Simple Linear Regression) bằng công thức đại số.

Listing 3.5: Đóng gói thuật toán Simple Linear Regression

```

1 class SimpleLinearRegression:
2     def __init__(self):
3         self.w0 = 0
4         self.w1 = 0
5
6     def fit(self, X, y):
7         n = len(X)
8         mean_x = sum(X) / n
9         mean_y = sum(y) / n
10
11        numerator = sum((X[i] - mean_x) * (y[i] - mean_y) for i in
12                        range(n))
13        denominator = sum((X[i] - mean_x) ** 2 for i in range(n))
14
15        if denominator == 0:

```

```

15         raise ValueError("Bao loi: Phuong sai cua X bang 0, khong
16             the tinh toan.")
17
18     self.w1 = numerator / denominator
19     self.w0 = mean_y - (self.w1 * mean_x)
20     return self
21
22     def predict(self, X):
23         return [(self.w1 * x_i + self.w0) for x_i in X]
24
25     def evaluate(self, X, y):
26         predictions = self.predict(X)
27         mse = sum((predictions[i] - y[i]) ** 2 for i in range(len(y)))
28             ) / len(y)
29         return mse

```

3.6 Mẫu thiết kế Pipeline

Pipeline là lớp quản lý chuỗi các bước biến đổi dữ liệu nối tiếp nhau, cho phép tự động hóa toàn bộ quy trình tiền xử lý trước khi đưa vào mô hình dự đoán.

Listing 3.6: Triển khai cấu trúc Pipeline cơ bản

```

1 class Pipeline:
2     def __init__(self, steps):
3         self.steps = steps
4
5     def fit_transform(self, arr):
6         data = arr
7         for transformer in self.steps:
8             data = transformer.fit_transform(data)
9         return data
10
11     def transform(self, arr):
12         data = arr
13         for transformer in self.steps:
14             data = transformer.transform(data)
15         return data

```

3.7 Tóm tắt chương

- Khái niệm Lớp, Đối tượng và tư duy lập trình hướng đối tượng.
- Ứng dụng các Magic Methods (`__init__`, `__call__`) trong xây dựng Framework.
- Nguyên lý Kế thừa giúp tái sử dụng và quản lý mã nguồn linh hoạt.
- Đóng gói thuật toán Machine Learning thành Class theo giao diện tiêu chuẩn.
- Phân biệt rõ ràng siêu tham số và tham số học được thông qua các thuộc tính của đối tượng

Chương 4: Hồi quy tuyến tính (Linear Regression)

Hồi quy tuyến tính là một trong những thuật toán học có giám sát (Supervised Learning) cơ bản và phổ biến nhất trong Machine Learning, dùng để dự đoán giá trị đầu ra liên tục.

4.1 Mô hình Hồi quy tuyến tính đơn biến

Định nghĩa 4.1 (Hồi quy tuyến tính đơn biến). Cho tập dữ liệu $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Mô hình biểu diễn mối quan hệ giữa biến độc lập x và biến phụ thuộc y theo dạng tuyến tính:

$$y = w_0 + w_1x + \epsilon \quad (4.1)$$

trong đó w_0 là hệ số chặn (intercept), w_1 là hệ số góc (slope), và ϵ là sai số ngẫu nhiên.

4.2 Hàm mất mát (Loss Function)

Để tìm các hệ số w_0, w_1 tối ưu, người ta sử dụng sai số bình phương trung bình (Mean Squared Error - MSE):

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (y_i - (w_0 + w_1x_i))^2 \quad (4.2)$$

Mục tiêu của bài toán là tìm (w_0, w_1) sao cho $J(w_0, w_1)$ đạt giá trị nhỏ nhất.

4.3 Cài đặt Hồi quy tuyến tính với Scikit-Learn

Dưới đây là đoạn code Python minh họa việc xây dựng mô hình Linear Regression trên tập dữ liệu thực tế:

Listing 4.1: Thực thi Linear Regression bằng Scikit-Learn

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3
4 # 1. Tạo dữ liệu giả định
5 X = np.array([[1], [2], [3], [4], [5]])
6 y = np.array([2.1, 3.9, 6.1, 8.2, 9.9])
7
```



```
8 # 2. Khởi tạo và huấn luyện mô hình
9 model = LinearRegression()
10 model.fit(X, y)
11
12 # 3. In kết quả trong số  $w_0$  và  $w_1$ 
13 print("Hệ số  $w_1$  (slope):", model.coef_[0])
14 print("Hệ số  $w_0$  (intercept):", model.intercept_)
15
16 # 4. Dự đoán giá trị mới
17 y_pred = model.predict([[6]])
18 print("Dự đoán cho X=6:", y_pred[0])
```

4.4 Tóm tắt chương

- Bản chất mô hình Hồi quy tuyến tính và bài toán tối ưu hàm MSE.
- Phương pháp hạ độ dốc (Gradient Descent) để cập nhật trọng số.
- Thao tác huấn luyện mô hình bằng thư viện `scikit-learn`.